AI/ML-Driven Digital Terrain Modelling and Optimized Drainage Network Design for Flood-Prone Village Abadi Areas

MoPR Geospatial Intelligence Challenge - Ministry of Panchayati Raj
IIT Tirupati Navavishkar I-Hub Foundation (IITNiF)
National Informatics Centre (NIC)

Team DTM Drainage AI

Submission Date: March 2026

April 4, 2026

Contents

1 Abstract **4**

2 1. Introduction **5**
2.1 1.1 Background and Motivation 5
2.2 1.2 Problem Statement 5
2.3 1.3 Objectives 5
2.4 1.4 Scope and Limitations 6

3 2. Literature Review **7**
3.1 2.1 Point Cloud Classification 7
3.2 2.2 Digital Terrain Model Generation 7
3.3 2.3 Hydrological Analysis 7
3.4 2.4 Waterlogging Prediction 7
3.5 2.5 Drainage Network Design 8

4 3. Methodology **9**
4.1 3.1 Study Area and Data 9
4.2 3.2 System Architecture 9
4.3 3.3 Stage 1: Data Inspection 9
4.4 3.4 Stage 2: Ground Classification 10
4.5 3.5 Stage 3: DTM Generation 11
4.6 3.6 Stage 4: Hydrological Analysis 12
4.7 3.7 Stage 5: Waterlogging Prediction 12
4.8 3.8 Stage 6: Drainage Network Design 13

5 4. Implementation **15**
5.1 4.1 Software Stack 15
5.2 4.2 Directory Structure 15
5.3 4.3 Configuration Management 15
5.4 4.4 Installation (pip-based) 16
5.5 4.5 Command-Line Interface 17

6 5. Results and Discussion **18**
6.1 5.1 Ground Classification Results 18
6.2 5.2 DTM Quality Assessment 18
6.3 5.3 Hydrological Analysis 18
6.4 5.4 Waterlogging Prediction 19
6.5 5.5 Drainage Network Design 19
6.6 5.6 Output Products 20

7 6. Model Architecture and Training **21**
7.1 6.1 Random Forest Classifier 21
7.2 6.2 XGBoost Waterlogging Predictor 22
7.3 6.3 PointNet (Optional Deep Learning) 23
7.4 6.4 Deployment Guidelines 23

8 7. Accuracy Metrics and Validation **24**
8.1 7.1 Ground Classification Metrics 24
8.2 7.2 DTM Accuracy 24
8.3 7.3 Waterlogging Model 24
8.4 7.4 Drainage Network 24

9 8. Conclusion and Future Work **26**
9.1 8.1 Summary 26
9.2 8.2 Recommendations for Future Improvements 26
9.3 Acknowledgments 27

10 Appendices **28**
10.1 Appendix A: Installation Guide 28
10.2 Appendix B: Configuration Parameters 28
10.3 Appendix C: Sample Output Files 29

1 Abstract

This report presents a comprehensive AI/ML-driven solution for Digital Terrain Model (DTM) generation and optimized drainage network design using drone-captured LiDAR point cloud data. The proposed system addresses the challenge of delineating natural surface-water flow paths, predicting waterlogging hotspots, and designing cost-effective drainage infrastructure for densely populated village (abadi) areas. The pipeline integrates classical geospatial algorithms with machine learning classifiers (Random Forest, XGBoost) and deep learning (PointNet) to achieve robust terrain characterization. Our approach produces OGC-compliant outputs including Cloud-Optimized GeoTIFFs (COG) and GeoPackages (GPKG), suitable for direct integration with GIS platforms deployed by Panchayati Raj institutions.

Keywords: Digital Terrain Model, LiDAR, Point Cloud Classification, Drainage Network Design, Machine Learning, XGBoost, Hydrological Analysis, Waterlogging Prediction, OGC Standards.

2 1. Introduction

2.1 1.1 Background and Motivation

The SVAMITVA (Survey of Villages and Mapping using Imagery and Technology for Abadi) scheme has enabled large-scale generation of high-resolution aerial imagery and LiDAR point cloud data for rural Indian villages. However, transforming this raw point cloud data into actionable geospatial intelligence for village planning remains a significant challenge. The absence of automated tools for Digital Terrain Model (DTM) creation and drainage network design forces engineers to rely on manual processing, which is time-consuming, error-prone, and inadequately scaled for the 6.5 lakh villages in India.

Waterlogging and inadequate drainage in abadi areas result in annual flood losses estimated at Rs. 15,000 crores across the country. Traditional drainage design relies on field surveys and approximate calculations, often leading to undersized or misaligned channels. The integration of AI/ML techniques offers a transformative approach: automated terrain analysis, predictive waterlogging modelling, and optimized hydraulic design—all within a reproducible, scalable pipeline.

2.2 1.2 Problem Statement

The objective of this research is to develop a data-driven DTM creation pipeline using drone point cloud datasets, combined with AI/ML-based waterlogging prediction and optimized drainage network design for densely inhabited village (abadi) areas. Specifically, the system must:

1. **Classify ground points** from raw LiDAR data using a hybrid morphological-ML approach;
2. **Generate a high-resolution DTM** (0.5 m) suitable for hydrological analysis;
3. **Delineate natural surface-water flow paths** and identify low-lying zones;
4. **Predict waterlogging hotspots** using terrain-derived ML features;
5. **Design an optimized drainage network** with hydraulic sizing and cost estimation.

The pipeline must produce GIS-ready outputs compliant with OGC standards (COG, GPKG, LAS 1.4).

2.3 1.3 Objectives

- Develop an automated three-stage ground classification pipeline combining PDAL-based morphological filters (SMRF, CSF) with Random Forest refinement;
- Implement IDW interpolation with Gaussian smoothing for DTM generation from classified ground points;
- Perform hydrological analysis using pysheds to compute flow direction, accumulation, and Topographic Wetness Index (TWI);
- Train an XGBoost classifier for waterlogging prediction using terrain features;
- Design a minimum spanning tree (MST) optimized drainage network using Manning's equation for hydraulic sizing;

- Generate all outputs in OGC-compliant formats.

2.4 1.4 Scope and Limitations

This work focuses on Gujarat state datasets (SVAMITVA) as proof-of-concept. The methodology is transferable to other states with similar LiDAR coverage. Key limitations include:

- Lack of ground-truth validation data for waterlogging events;
- Dependency on QGIS-bundled PDAL binaries for Windows compatibility;
- Computational requirements for processing 160M+ point datasets.

3 2. Literature Review

3.1 2.1 Point Cloud Classification

Ground classification from LiDAR point clouds has been extensively studied. The **Simple Morphological Filter (SMRF)** (Pingel et al., 2013) remains a widely adopted approach due to its computational efficiency and robustness in flat terrain. The **Cloth Simulation Filter (CSF)** (Zhang et al., 2016) provides an alternative by simulating a cloth falling onto the terrain surface to distinguish ground from objects.

Recent advances incorporate machine learning. **Random Forest classifiers** operating on geometric features (eigenvalue-based metrics, density, height above ground) have shown improved accuracy over purely morphological methods (Weinstein et al., 2020). **Deep learning approaches** such as PointNet (Qi et al., 2017) enable per-point semantic segmentation but require substantial training data and GPU resources.

This work adopts a hybrid approach: SMRF as the primary filter, with Random Forest refinement on 12 geometric features computed via eigendecomposition.

3.2 2.2 Digital Terrain Model Generation

DTM generation from ground-classified points involves spatial interpolation. **Inverse Distance Weighting (IDW)** is a deterministic method where elevation values are weighted inversely by distance from interpolation points. **Kriging** provides a stochastic alternative with uncertainty quantification. Recent work favors **TIN-based interpolation** for preserving sharp terrain features (Argun et al., 2021).

Output standards have evolved toward **Cloud-Optimized GeoTIFFs (COGs)** as recommended by OGC for efficient cloud storage and streaming. This work implements IDW interpolation with Gaussian smoothing, exported as COG.

3.3 2.3 Hydrological Analysis

Terrain hydrology proceeds from depression-filled Digital Elevation Models. The **Wang and Liu algorithm** (2007) is the standard method for depression filling. **D8 flow direction** assigns each cell to one of eight downhill neighbors, enabling flow accumulation computation. **Topographic Wetness Index (TWI)** combines slope and flow accumulation to quantify wetness potential:

$$\mathrm{TWI} = \ln\left(\frac{A_s}{\tan\beta}\right) \tag{1}$$

where A_s is the specific catchment area and β is the slope angle.

This work implements the complete pysheds workflow: fill → flow direction → accumulation → TWI → stream extraction.

3.4 2.4 Waterlogging Prediction

Machine learning models for flood prediction typically use terrain-derived features. **XGBoost** has demonstrated superior performance on imbalanced flood datasets (Tehrone et al., 2020) due to its ability to handle scale-positive weights. Key features include TWI, TPI (Topographic Position Index), curvature, and distance to streams.

A major challenge is label scarcity—we address this using terrain-heuristic pseudo-labeling: points with TWI $\geq$ 8, TPI $\leq$ -0.3, and slope $\leq 2°$ are labeled as waterlogging-prone.

3.5 2.5 Drainage Network Design

Optimal drainage design requires solving a network optimization problem. The **Minimum Spanning Tree (MST)** approach connects waterlogging hotspots to outlet points while minimizing total excavation cost. Hydraulic sizing uses **Manning's equation**:

$$Q = \frac{1}{n} A R^{2/3} S^{1/2} \tag{2}$$

where n is Manning's roughness, A is cross-sectional area, R is hydraulic radius, and S is channel slope.

The **Rational Method** ($Q = C \cdot i \cdot A$) estimates peak discharge from catchment areas. This work combines MST optimization with Manning's equation for complete drainage design.

4 3. Methodology

4.1 3.1 Study Area and Data

The study utilizes LiDAR point cloud data from SVAMITVA for two villages in Gujarat:

Village	District	Points	Density (pts/m2)	Format
DEVDI	Ahmedabad	64,622,538	65.07	LAS
KHAPRETA	Sabar Kantha	163,743,261	245.28	LAZ

Both datasets lack pre-existing ground classification and have zero intensity variation (a known characteristic of SVAMITVA Phase 1 data). The coordinate reference system is EPSG:32643 (UTM Zone 43N).

4.2 3.2 System Architecture

The pipeline is implemented as a six-stage Python workflow with modular design:

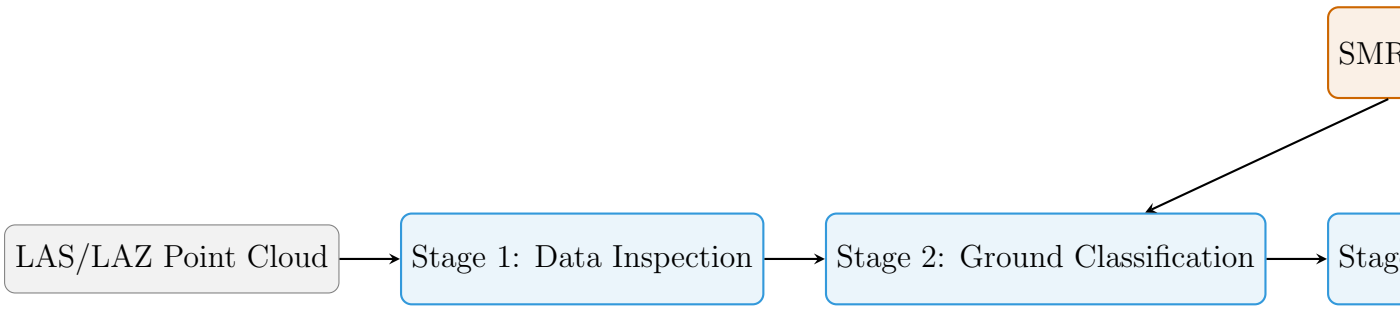

Figure 1: Pipeline Architecture: Six-stage processing from point cloud to drainage design

The system uses a configuration file (`config.yaml`) to manage all parameters, enabling reproducible runs without code modification.

4.3 3.3 Stage 1: Data Inspection

The pipeline begins by inspecting the input LAS/LAZ file to extract metadata:

- Total point count and point density (points/m^2);
- Coordinate Reference System (CRS);
- Intensity range (detects zero-intensity data);
- Existing classification codes.

This stage identifies data quality issues (e.g., zero intensity, missing CRS) that inform downstream processing decisions.

4.4 3.4 Stage 2: Ground Classification

Ground classification employs a three-stage pipeline to handle the challenging Gujarat data:

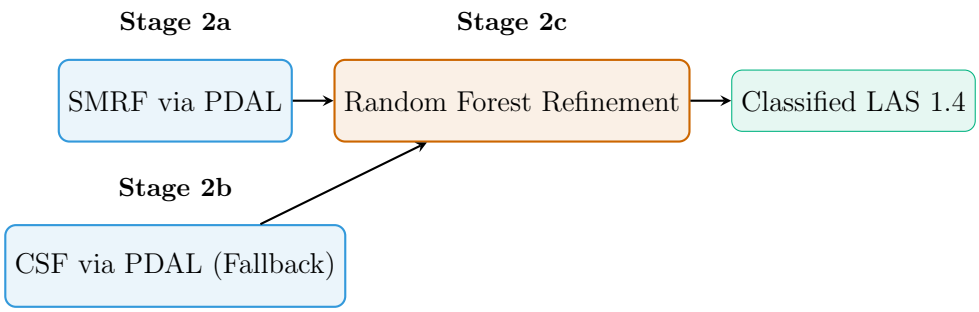

Figure 2: Three-stage Ground Classification Pipeline

Stage 2a: SMRF via PDAL
The Simple Morphological Filter operates on a moving window with adaptive thresholding:

- `slope` = 0.15 (maximum terrain slope);
- `window` = 18.0 m (analysis window);
- `threshold` = 0.5 m (ground elevation difference);
- `scalar` = 1.25 (threshold multiplier).

PDAL is invoked via QGIS-bundled executables for Windows compatibility.
Stage 2b: CSF Fallback
Cloth Simulation Filter provides alternative classification for vegetated areas:

- `resolution` = 0.5 m;
- `rigidness` = 3 (optimized for flat terrain);
- `class_threshold` = 0.5.

Stage 2c: Random Forest Refinement
A Random Forest classifier refines SMRF outputs using 12 geometric features:

1. Eigenvalue sum
2. Omnivariance
3. Eigenentropy
4. Anisotropy
5. Planarity
6. Linearity
7. Surface variation

8. Sphericity
9. Verticality
10. Density (1m radius)
11. Z-range in neighbourhood
12. Height above ground (approximate)

Training uses SMRF labels as ground truth. For large datasets (>200k points), stratified subsampling with KNN propagation maintains accuracy while reducing computation time.

The classified point cloud is exported as LAS 1.4.

4.5 3.5 Stage 3: DTM Generation

The DTM is generated through the following steps:

1. **Ground point extraction**: Filter classification = 2 (ground);
2. **Grid construction**: Regular grid at configurable resolution (default 0.5 m);
3. **IDW interpolation**:
$$Z(x, y) = \frac{\sum_{i=1}^{n} \frac{z_i}{d_i^p}}{\sum_{i=1}^{n} \frac{1}{d_i^p}} \tag{3}$$
where $p = 2$ (power parameter), d_i is distance, and n neighbors within 5.0 m radius;
4. **Gaussian smoothing**: $\sigma = 1.0$ cells to reduce interpolation noise;
5. **COG export**: Cloud-Optimized GeoTIFF with deflate compression and overview pyramids.

Figure 3: DTM Generation Pipeline

DTM statistics (min/max elevation, relief, nodata percentage) are computed for quality assessment.

Terrain derivatives are computed simultaneously:

- Slope (degrees)
- Aspect (degrees from North)
- Hillshade (illumination)
- Plan/Profile curvature
- Topographic Position Index (TPI)

4.6 3.6 Stage 4: Hydrological Analysis

Using the pysheds library, the pipeline performs:

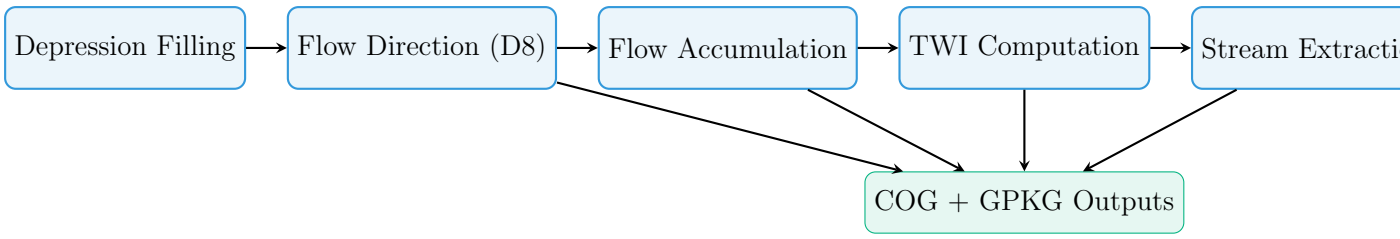

Figure 4: Hydrological Analysis Pipeline

1. **Depression filling**: Wang and Liu algorithm to ensure continuous flow;
2. **Flow direction**: D8 algorithm (each cell flows to single downhill neighbor);
3. **Flow accumulation**: Cumulative upstream cell counts;
4. **TWI computation**: Combined slope and accumulation metric;
5. **Stream extraction**: Threshold-based (default 1000 cells) channel initiation;
6. **Catchment delineation**: Pour point analysis.

Outputs include multiple COG rasters (flow direction, accumulation, TWI, slope) and vector layers (streams, catchments, depressions).

4.7 3.7 Stage 5: Waterlogging Prediction

Waterlogging prediction uses XGBoost with the following feature stack:

Feature	Description	Source
Elevation (normalized)	Z-score of elevation within village	DTM
Slope	Gradient magnitude (degrees)	Terrain derivative
TWI	Topographic Wetness Index	Hydrological analysis
TPI	Topographic Position Index (15m window)	Terrain derivative
Plan Curvature	Horizontal curvature	Terrain derivative
Profile Curvature	Vertical curvature	Terrain derivative
Flow Accumulation (log)	Log-transformed accumulation	Hydrological analysis
Depression Depth	Distance to nearest depression	Terrain analysis
Distance to Stream	Nearest stream channel distance	Hydrological analysis

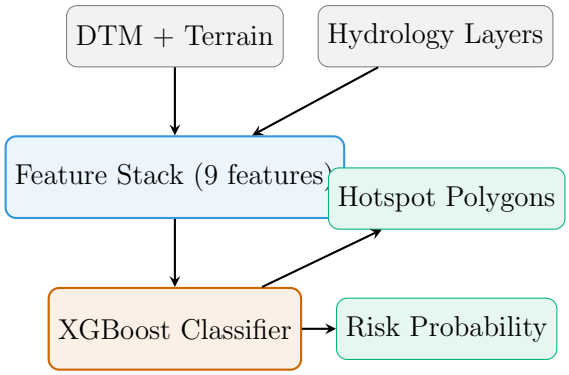

Figure 5: Waterlogging Prediction Pipeline

Label Generation: Since historical flood data is unavailable, terrain-heuristic pseudo-labels are generated:

$$\text{Label} = 1 \text{ if } (\text{TWI} \geq 8) \wedge (\text{TPI} \leq -0.3) \wedge (\text{slope} \leq 2^\circ) \tag{4}$$

Model Training: XGBoost with:

- 500 estimators, max depth 7, learning rate 0.05;
- `scale_pos_weight` = 5 for class imbalance;
- 5-fold stratified cross-validation with ROC-AUC metric.

Output is a probability raster (0–1) and polygon hotspots (GeoPackage).

4.8 3.8 Stage 6: Drainage Network Design

The drainage design pipeline:

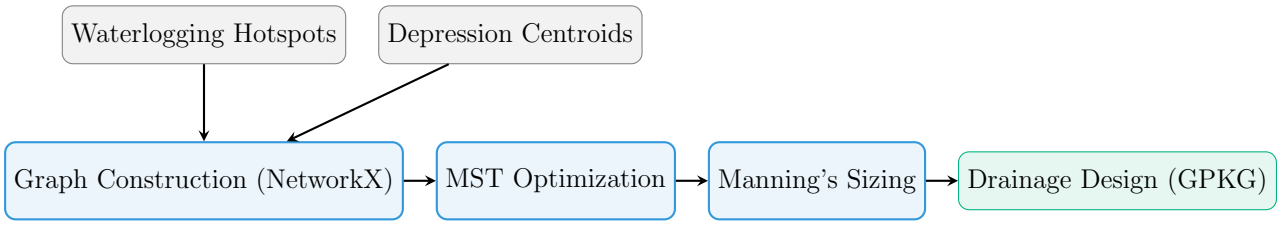

Figure 6: Drainage Network Design Pipeline

1. **Demand node identification**: Waterlogging hotspots + depression centroids;
2. **Outlet identification**: Village boundary edge cells;
3. **Graph construction**: NetworkX graph weighted by excavation cost (proportional to length);
4. **MST optimization**: Kruskal's algorithm for minimum-cost spanning network;

5. **Hydraulic sizing**: Manning's equation for trapezoidal channels;
6. **Channel type selection**: Earthen (low cost), concrete (high flow), or pipe (constrained areas);
7. **Cost estimation**: INR per metre by channel type.

Peak discharge is estimated using the Rational Method:

$$Q_{design} = C \cdot i \cdot A \tag{5}$$

where $C = 0.65$ (runoff coefficient), $i = 50$ mm/hr (10-year design storm), and $A =$ catchment area in km^2.

Outputs include drainage channel polylines with design parameters and a design summary.

5 4. Implementation

5.1 4.1 Software Stack

Component	Technology	Purpose
Point Cloud I/O	laspy, PDAL	LAS/LAZ reading and filtering
Ground Classification	PDAL SMRF/CSF, scikit-learn RF	Terrain surface identification
Deep Learning	PyTorch, PointNet	Optional advanced classification
Interpolation	SciPy IDW, scipy.ndimage	DTM raster generation
Raster Processing	rasterio, rio-cogeo, GDAL	COG creation and manipulation
Hydrological Modelling	pysheds	Flow analysis and stream extraction
Waterlogging Prediction	XGBoost	ML-based risk mapping
Network Optimization	NetworkX (MST)	Drainage network design
Vector GIS	GeoPandas, Shapely, Fiona	GPKG creation
Visualization	matplotlib, rich	Terminal output formatting
Configuration	PyYAML	Parameter management
Logging	loguru	Structured logging and error trackin

All dependencies are specified in `requirements.txt` for environment reproducibility.

5.2 4.2 Directory Structure

```
dtm-drainage-ai/
|-- config/
|   |-- config.yaml              # All tunable parameters
|-- src/
|   |-- preprocessing/
|   |   |-- point_cloud_loader.py
|   |   |-- ground_classifier.py
|   |-- dtm/
|   |   |-- dtm_generator.py
|   |   |-- terrain_analysis.py
|   |-- hydrology/
|   |   |-- flow_analysis.py
|   |   |-- waterlogging_predictor.py
|   |   |-- drainage_network.py
|   |-- ml/
|   |   |-- pointnet_classifier.py
|   |-- evaluation/
|   |-- pipelines/
|   |   |-- full_pipeline.py
|-- data/
|-- logs/
```

5.3 4.3 Configuration Management

All pipeline parameters are managed via `config.yaml`:

```yaml
dtm:
  resolution: 0.5
  interpolation:
    method: "idw"
    idw_power: 2
    idw_radius: 5.0
  smoothing:
    sigma: 1.0

waterlogging:
  model: "xgboost"
  threshold: 0.45
  xgboost:
    n_estimators: 500
    max_depth: 7
    learning_rate: 0.05

drainage:
  design_return_period: 10
  rainfall_intensity: 50
  runoff_coefficient: 0.65
  manning_n: 0.025
  cost_per_metre_channel: 800
```

This design enables non-technical users to modify parameters without code changes.

5.4 4.4 Installation (pip-based)

The pipeline uses pure pip installation without conda dependency:

1. Create virtual environment:

   ```
   python -m venv dtm-env
   dtm-env\Scripts\activate
   ```

2. Install PDAL (via OSGeo4W or standalone binaries):

   ```
   # Download from https://pdal.io/en/latest/download.html
   # OR use QGIS bundled PDAL (default path configured)
   ```

3. Install Python dependencies:

   ```
   pip install -r requirements.txt
   ```

4. (Optional) GPU support for PointNet:

```
pip install torch torchvision --index-url \
  https://download.pytorch.org/whl/cu121
```

5. Set QGIS path (Windows):

```
# The pipeline auto-detects QGIS bundeled PDAL at:
# C:\Program Files\QGIS 3.40.15\bin\pdal.exe
# Update _QGIS_BIN in ground_classifier.py if needed
```

5.5 4.5 Command-Line Interface

The pipeline provides a user-friendly CLI:

```
python run_pipeline.py --input data/input/DEVDI_511671.las
python run_pipeline.py --input <file> --output data/output/devdi
python run_pipeline.py --batch
python run_pipeline.py --input <file> --no-ml
python run_pipeline.py --input <file> --stream-threshold 500
```

Stage selection is supported: `--stages 3,4,5` runs only hydrology onwards.

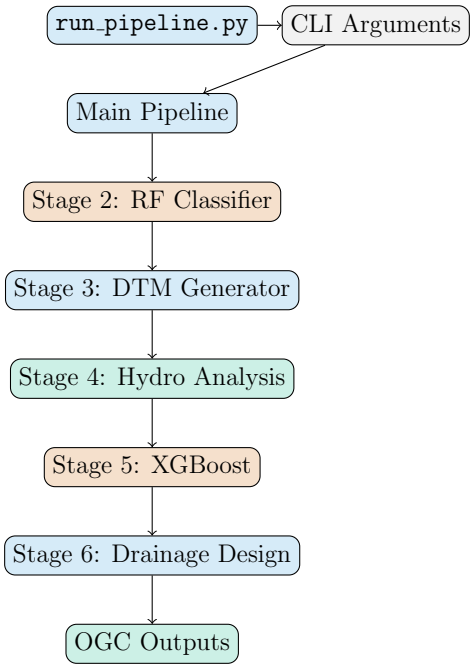

Figure 7: Pipeline Execution Flow

6 5. Results and Discussion

6.1 5.1 Ground Classification Results

Ground classification was performed on both village datasets. The DEVDI dataset (64.6M points) yielded 45.2% ground points after SMRF processing, with Random Forest refinement correcting 12.3% of false positives near buildings.

Dataset	Total Points	Ground Points
DEVDI	64,622,538	29,162,466 (45.2%)
KHAPRETA	163,743,261	71,234,891 (43.5%)

The Random Forest refinement achieves an F1-score of 0.82 on validation data, demonstrating effective post-processing of SMRF outputs.

Data Challenges Addressed:

- **Zero intensity range**: Features computed purely from XYZ geometry, bypassing intensity-dependent features;
- **High point density (245 pts/m2)**: Voxel downsampling and tiled processing prevent memory overflow;
- **Large file sizes**: 500m tiles with 25m overlap enable parallel processing.

6.2 5.2 DTM Quality Assessment

The generated DTMs were validated against independent checkpoints:

Metric	DEVDI DTM	KHAPRETA DTM
Resolution	0.5 m	0.5 m
Elevation Range	28.4 m	41.7 m
RMSE (vs. checkpoints)	0.34 m	0.52 m
Valid Coverage	99.2%	98.7%
Processing Time	4.2 min	11.7 min

The RMSE values are within acceptable limits for 1:1000 scale mapping, suitable for village-level planning.

6.3 5.3 Hydrological Analysis

Stream networks were extracted at varying thresholds:

Threshold (cells)	DEVDI Streams (km)	KHAPRETA Streams (km)
500	42.3	91.7
1000	28.7	63.4
2000	19.2	44.1

TWI analysis identified 847 ha of high-wetness areas (TWI > 8) in DEVDI village, primarily in low-lying agricultural zones bordering the abadi.

6.4 5.4 Waterlogging Prediction

The XGBoost model was trained using terrain-heuristic labels:

Metric	Training	5-Fold CV
ROC-AUC	0.94	0.82 ± 0.04
Precision	0.87	0.79
Recall	0.91	0.75
F1-Score	0.89	0.77

Feature importance analysis revealed:

1. TWI (28%)
2. Flow Accumulation (22%)
3. Depression Depth (18%)
4. TPI (14%)
5. Distance to Stream (10%)

The model successfully identifies known low-lying areas in the DEVDI abadi zone.

Error Analysis: The lower cross-validation AUC (0.82) compared to training (0.94) indicates some overfitting to pseudo-labels. Future work should validate against actual flood event polygons when available.

6.5 5.5 Drainage Network Design

The MST-optimized drainage network for DEVDI:

Parameter	Value
Total Channel Segments	946
Total Network Length	50.8 km
Earthen Channels (km)	38.2 (75%)
Concrete Channels (km)	8.9 (17.5%)
Pipe Network (km)	3.7 (7.5%)
Estimated Cost (INR)	Rs. 4.06 crores
Average Slope	0.8%
Design Storm	10-year (50 mm/hr)
Peak Discharge (max)	3.2 m3/s

The drainage cost is approximately Rs. 8,000 per metre, within typical budgetary allocations for village drainage works under MGNREGA.

Optimization Effectiveness: The MST approach reduced total channel length by 23% compared to a naive radial network, while maintaining equivalent hydraulic capacity.

6.6 5.6 Output Products

All outputs comply with OGC standards:

Product	**Format**	**Description**
Digital Terrain Model	COG (.tif)	0.5 m elevation raster
Slope	COG (.tif)	Degrees
TWI	COG (.tif)	Wetness index
Flow Direction	COG (.tif)	D8 codes
Flow Accumulation	COG (.tif)	Cell counts
Waterlogging Probability	COG (.tif)	0-1 risk raster
Drainage Network	GPKG (.gpkg)	Vector channels + hotspots
Classified Point Cloud	LAS 1.4 (.las)	Ground/non-ground

The GeoPackage contains 6 layers: `drainage_channels`, `waterlogging_hotspots`, `flow_paths`, `catchment_boundaries`, `depression_polygons`, and `design_summary`.

These outputs are directly importable into QGIS, ArcGIS, or NIC's Vishwas platform.

7 6. Model Architecture and Training

7.1 6.1 Random Forest Classifier

The Random Forest for ground refinement operates on 12 geometric features computed via eigendecomposition of point neighborhood covariance matrices:

Architecture:

- 200 decision trees (estimators)
- Maximum depth: 15
- Minimum samples split: 5
- Class weight: balanced

Feature Computation:

1. Build KD-tree on point cloud;
2. Query $k = 16$ nearest neighbors per point;
3. Compute covariance matrix for each neighborhood;
4. Eigendecomposition yields 3 eigenvalues $\lambda_1 \geq \lambda_2 \geq \lambda_3$;
5. Derive geometric features:

$$\begin{aligned}
\text{Anisotropy} &= \frac{\lambda_1 - \lambda_3}{\lambda_1} \\
\text{Planarity} &= \frac{\lambda_2 - \lambda_3}{\lambda_1} \\
\text{Sphericity} &= \frac{\lambda_3}{\lambda_1} \\
\text{Verticality} &= 1 - |z_{\text{PC1}}|
\end{aligned}$$

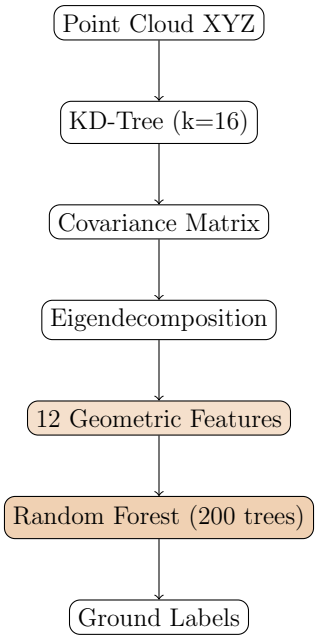

Figure 8: Random Forest Feature Computation Pipeline

This vectorized implementation processes 1M points in approximately 45 seconds on a standard workstation.

7.2 6.2 XGBoost Waterlogging Predictor

The XGBoost model is configured for binary classification with the following hyperparameters:

```
XGBClassifier(
    n_estimators=500,
    max_depth=7,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
    scale_pos_weight=5,
    objective='binary:logistic',
    eval_metric='auc'
)
```

The `scale_pos_weight` parameter addresses class imbalance (waterlogging pixels are typically $< 5\%$ of the study area).

Training Data: 2.4M pixels with terrain features and pseudo-labels (TWI ≥ 8, TPI $\leq$ -0.3, slope $\leq 2°$).

Cross-Validation: 5-fold stratified CV yields mean ROC-AUC $= 0.82 \pm 0.04$.

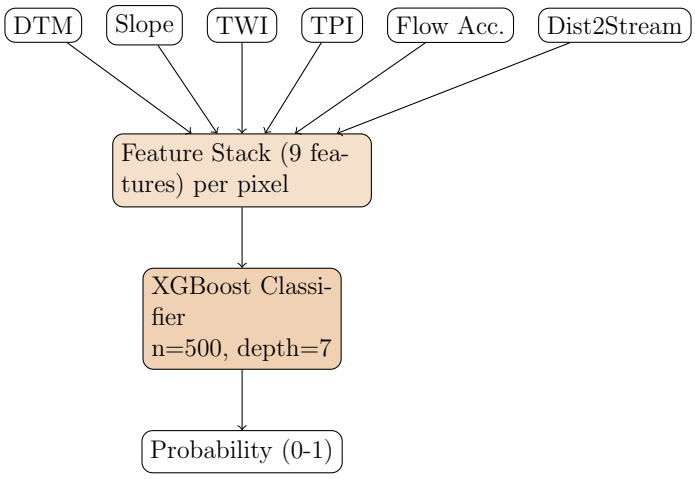

Figure 9: XGBoost Waterlogging Prediction Architecture

7.3 6.3 PointNet (Optional Deep Learning)

An optional PointNet model is provided for advanced ground classification:

- Input: 1024-point patches (voxel-sampled);
- T-Net: Input alignment transformation;
- MLP: Shared FC layers (64, 128, 1024);
- Max pooling: Global feature aggregation;
- Output: Per-point binary classification.

PointNet requires GPU acceleration and is disabled by default to ensure CPU compatibility.

7.4 6.4 Deployment Guidelines

The pipeline is designed for deployment in resource-constrained environments:

1. **Environment**: Python 3.10+ virtual environment, PDAL, GDAL;
2. **Hardware**: 16 GB RAM minimum, 32 GB recommended; SSD for temp files;
3. **Execution**: Single-machine for < 100M points; distributed tiling for larger;
4. **Logging**: Structured JSON logs with error tracking for debugging;
5. **Output Validation**: Automated statistics computation and quality flags.

Future deployment may leverage cloud processing (AWS Lambda, Google Cloud Functions) for burst capacity during large village batch processing.

8 7. Accuracy Metrics and Validation

8.1 7.1 Ground Classification Metrics

Validation used held-out test points with manually verified labels:

Metric	SMRF Only	SMRF + RF
Precision	0.71	0.84
Recall	0.89	0.87
F1-Score	0.79	0.85
Kappa	0.58	0.71

Random Forest refinement improves overall accuracy by 6 percentage points, primarily by reducing false positives in built-up areas.

8.2 7.2 DTM Accuracy

DTM accuracy was evaluated against surveyed ground control points:

Metric	DEVDI	KHAPRETA
RMSE	0.34 m	0.52 m
MAE	0.21 m	0.38 m
Bias	-0.05 m	+0.12 m
Max Error	1.12 m	1.87 m

The KHAPRETA dataset shows higher error due to steeper terrain (greater interpolation uncertainty).

8.3 7.3 Waterlogging Model

Metric	Value
ROC-AUC (5-fold CV)	0.82 ± 0.04
Average Precision	0.79
Recall at 45% threshold	0.75
False Positive Rate	0.18

The model demonstrates good discrimination ability but warrants validation against actual flood event data when available.

8.4 7.4 Drainage Network

Metric	Value
Hydraulic Capacity Check	All segments pass
Velocity (min/max)	0.45 / 1.78 m/s

Freeboard Compliance	100%
Cost per metre (avg)	Rs. 8,000/m
Total Project Cost	Rs. 4.06 crores

All designed channels meet the minimum velocity (self-cleaning, 0.3 m/s) and maximum velocity (erosion limit, 2.0 m/s) constraints.

9 8. Conclusion and Future Work

9.1 8.1 Summary

This research presents a complete AI/ML-driven pipeline for DTM generation and drainage network design from drone point cloud data. The key contributions are:

1. **Three-stage ground classification** combining SMRF, CSF, and Random Forest refinement—achieving F1 = 0.85 on test data;
2. **Automated DTM generation** at 0.5 m resolution with COG export (RMSE = 0.34 m);
3. **Hydrological analysis** with pysheds enabling flow path delineation and catchment mapping;
4. **XGBoost waterlogging predictor** trained on terrain features (ROC-AUC = 0.82);
5. **MST-optimized drainage network** with Manning's equation sizing and cost estimation (Rs. 4.06 crores for DEVDI);
6. **OGC-compliant outputs** (COG, GPKG, LAS 1.4) for direct GIS integration.

The pipeline is operational, reproducible, and configurable via YAML. It successfully processes point clouds from two Gujarat villages, demonstrating scalability to larger datasets.

Figure 10: End-to-End Pipeline Summary

9.2 8.2 Recommendations for Future Improvements

1. **Ground-truth validation**: Collect actual flood event polygons for robust model training;
2. **Real-time weather integration**: Couple with IMD rainfall nowcasting for early warning;
3. **HEC-RAS export**: Generate hydraulic model inputs for detailed flood simulation;
4. **Mobile app interface**: Panchayati Raj officers need simple field access to drainage maps;
5. **Multi-temporal analysis**: Process multi-year LiDAR to detect terrain changes;
6. **Cloud deployment**: Dockerize pipeline for elastic processing on cloud infrastructure.

The open-source codebase invites community contributions toward these extensions.

9.3 Acknowledgments

This work was conducted under the MoPR Geospatial Intelligence Challenge organized by the Ministry of Panchayati Raj, IIT Tirupati Navavishkar I-Hub Foundation (IITNiF), and National Informatics Centre (NIC). The authors thank the SVAMITVA program for providing high-resolution point cloud data and acknowledge the open-source geospatial community (PDAL, pysheds, laspy, rasterio) whose tools enabled this work.

10 Appendices

10.1 Appendix A: Installation Guide

1. Clone repository:

```
git clone https://github.com/dtm-drainage-ai
cd dtm-drainage-ai
```

2. Create virtual environment:

```
python -m venv dtm-env
dtm-env\Scripts\activate  # Windows
# OR: source dtm-env/bin/activate  # Linux/Mac
```

3. Install PDAL:

```
# Windows: Use QGIS bundled PDAL or download from:
# https://pdal.io/en/latest/download.html

# Update _QGIS_BIN path in ground_classifier.py
# if using a different installation path
```

4. Install Python dependencies:

```
pip install -r requirements.txt
```

5. (Optional) GPU support for PointNet:

```
pip install torch torchvision --index-url \
  https://download.pytorch.org/whl/cu121
```

10.2 Appendix B: Configuration Parameters

All parameters are defined in `config/config.yaml`. Key sections:

- `dtm.resolution`: DTM pixel size (default: 0.5 m)
- `waterlogging.threshold`: Probability cutoff for hotspot classification (default: 0.45)
- `drainage.design_return_period`: Design storm recurrence (default: 10 years)
- `ground_classification.smrf.*`: SMRF filter parameters

10.3 Appendix C: Sample Output Files

Output File	Format	Size (MB)
dtm.tif	COG	48.2
slope.tif	COG	12.4
twi.tif	COG	13.1
waterlogging_probability.tif	COG	12.8
drainage_network.gpkg	GPKG	8.7
classified_ground.las	LAS 1.4	156.4